

THEORETICAL FRAMEWORK FOR Problem Solving

A Macro-Level Guide to Algorithmic Paradigms

At the highest philosophical level, computer science problem solving boils down to a fundamental dichotomy: determining whether a system is **predictable** or **chaotic**. By assessing a problem's constraints and rules, you can classify the solution into one of three macro-strategies — each representing a distinct mental model for attacking unknowns.

■ THE GOLDEN QUESTION

"Is this system predictable enough to construct an answer, or too chaotic, requiring a search?"

Answering this single question within the first 3 minutes of reading a problem will eliminate at least half of all possible solution families.

PARADIGM TAXONOMY AT A GLANCE

Family	Paradigm	Typical Complexity	Key Signal
Construct	Greedy	$O(N \log N) / O(N)$	Locally optimal = globally optimal
Construct	Constructive	$O(N) / O(N^2)$	"Output any valid..."
Construct	Math / Number Theory	$O(1) / O(\log N)$	Massive N (10^4), direct formula
Search	Complete Search	$O(2^N) / O(N!)$	Tiny N (≤ 20), brute force viable
Search	Binary Search on Answer	$O(\log N \cdot f(N))$	Verify is easy; optimize is hard
Search	Graph Traversal	$O(V + E)$	Nodes + edges, paths, connectivity
Smart Search	Dynamic Programming	$O(N^2) / O(N \cdot K)$	Overlapping subproblems, "# of ways"
Smart Search	Divide & Conquer	$O(N \log N)$	Disjoint subproblems, merge results

The CONSTRUCT Family

These approaches are deployed when a problem contains underlying mathematical properties, invariants, or logical rules that can be directly exploited. Instead of simulating state changes or exhaustively enumerating possibilities, you build the final answer in one decisive logical motion. The hallmark of this family is that the structure of the problem itself tells you the answer — you just need to be clever enough to read it.

◆ Greedy Algorithms

$O(N \log N)$ typ

CORE PHILOSOPHY

Make the locally optimal choice at each step with the certainty (provable) that these local optima accumulate into a global optimum. You commit greedily and never look back. The algorithm never reconsiders a past decision — which is what makes it fast, and what makes proving correctness the hardest part.

TIP: Proof technique: Exchange argument — assume optimal $\neq$ greedy, swap one choice, show you can only do $\geq$ as well.

TIP: Common failure mode: Mistaking a DP problem for greedy (e.g., 0/1 Knapsack). Test on small counter-examples first.

WHEN TO USE

- Problem exhibits **optimal substructure** — optimal solutions to sub-problems compose into the overall optimum.
- Problem has the **greedy choice property** — choosing the locally best option never "punishes" you in future steps.
- Constraints are large: $N \leq 10^5$ or 10^6 , requiring $O(N \log N)$ or better.
- The problem involves **scheduling, intervals, or resource allocation** with a natural ordering.
- Sorting the input immediately reveals the decision order.

EXAMPLES

- Activity Selection / Interval Scheduling — always pick the job ending soonest.
- Fractional Knapsack — take items by highest value-to-weight ratio.
- Huffman Coding — greedily merge the two lowest-frequency nodes.
- Minimum Spanning Tree (Kruskal's, Prim's) — always add the cheapest safe edge.
- Array minimal product — pair 1s with the largest numbers; always strictly optimal.
- Dijkstra's algorithm (non-negative weights) — greedy relaxation of shortest paths.

◆ Constructive / Ad-Hoc Algorithms

$O(N)$ to O

CORE PHILOSOPHY

There is no standard template to memorize. The solver must engage in pure logical deduction — reasoning about invariants, parity, pigeonhole constraints, or structural properties — to produce any valid configuration satisfying the problem's conditions. Creativity and mathematical maturity are the primary tools.

TIP: Strategy: Draw small cases ($N=3,4,5$), stare at patterns, conjecture a formula, prove it by induction.

TIP: If the answer is 'impossible', parity / coloring arguments are often the proof path.

WHEN TO USE

- Problem says "**output any valid array / matrix / string**" that satisfies conditions.
- Involves a game, sequence of moves, or very specific allowed operations.
- Small or medium N with seemingly complex interactions that have a hidden pattern.
- Parity / pigeonhole / symmetry arguments can guarantee existence of a solution.
- The problem asks you to prove impossibility or construct an adversarial counter-example.

EXAMPLES

- Construct a permutation of length N where no two adjacent elements sum to a prime.
- Place N non-attacking rooks on an $N \times N$ chessboard with additional parity constraints.
- Given N , build an array where every prefix has at least as many positives as negatives.
- Construct a magic square or balanced partition satisfying sum constraints.

◆ Math & Number Theory

$O(1)$ or $O(\log)$

CORE PHILOSOPHY

The answer can be computed **directly** using a closed-form formula, combinatorial identity, modular arithmetic, or geometric insight — no simulation required. When N reaches 10^{18} , even an $O(N)$ loop is fatal; the solution must reduce to $O(1)$ or $O(\log N)$ algebraic computation.

TIP: When you see $\text{mod } 10^9+7$, immediately think Fermat's little theorem for division.

TIP: Combinatorics on a grid almost always reduces to Pascal's triangle or Catalan numbers.

WHEN TO USE

- N is astronomically large (10^{15} – 10^{18}) — loops are impossible.
- Problem involves counting: combinations, permutations, partitions, paths on grids.
- Divisibility, GCD/LCM, prime factorization, or modular inverses appear.
- Geometry: intersections, areas, convex hull, or coordinate transformations.
- The answer must be given modulo a prime (typically 10^9+7 or 998244353).

EXAMPLES

→ Trailing zeros in $N!$ — count factors of 5: $\lfloor N/5 \rfloor + \lfloor N/25 \rfloor + \lfloor N/125 \rfloor + \dots$

→ $C(n, k) \bmod p$ — Lucas's theorem or Fermat's little theorem for modular inverse.

→ Number of paths in an $M \times N$ grid — combinatorics: $C(M+N-2, M-1)$.

→ Fast exponentiation (binary exponentiation) — $a^b \bmod m$ in $O(\log b)$.

→ Sieve of Eratosthenes — all primes up to N in $O(N \log \log N)$.

→ Extended Euclidean algorithm — solve $ax + by = \text{gcd}(a, b)$.

The SEARCH Family

When the rules of a problem interact in ways too complex to deduce a final state mathematically, you must rely on systematic exploration of a **state space**. Each state is a snapshot of the system at a point in time; transitions between states are governed by the problem's rules. The goal is to traverse this space efficiently, exploiting structure wherever possible.

◆ Complete Search (Brute Force / Backtracking)

 $O(2^N)$ or $O(N!)$

CORE PHILOSOPHY

Generate all possible configurations, test each against the constraints, and select the optimal valid result. This is the most honest algorithm: it makes no assumptions and misses nothing. Backtracking prunes branches that are guaranteed to violate constraints before fully expanding them, often yielding exponential practical speed-ups while maintaining worst-case completeness.

TIP: Always prune early: check constraints before recursing deeper, not after.

TIP: Iterative deepening DFS (IDDFS) achieves BFS-optimal depth with DFS memory efficiency.

WHEN TO USE

- $N \leq 20$: subset enumeration is $O(2^N)$, feasible.
- $N \leq 10-12$: permutation enumeration is $O(N!)$, feasible.
- No known polynomial algorithm exists (NP-hard recognition problems at small scale).
- Used as a **reference validator** to verify faster solutions on small inputs.
- Constraint propagation (e.g., Sudoku rules) makes pruning extremely effective.

EXAMPLES

- N-Queens — place N queens with no attacks; backtrack on conflicts.
- Generating all valid Sudoku solutions — constraint propagation + backtracking.
- Travelling Salesman Problem at $N \leq 15$ — bitmask DP or pure permutation search.
- All subsets of size K with a given property — recursive subset enumeration.
- Word break / partition into palindromes — enumerate all splits.

◆ Binary Search on the Answer

$O(N \log N)$ or $O(N \log \dots)$

CORE PHILOSOPHY

Reframe an optimization problem: instead of asking "what is the optimal score?" ask "is score X achievable?". If the feasibility predicate `isValid(X)` is monotonic (once X becomes impossible, everything harder also fails), binary search locates the exact boundary in $O(\log(\text{answer_range}))$ calls to `isValid` — each of which can be $O(N)$ or $O(N \log N)$.

TIP: The `isValid` function must be written first — if you can't write it cleanly, reconsider the approach.

TIP: Off-by-one errors are the #1 bug: use `lo=1`, `hi=max`, and think carefully about `lo<=hi` vs `lo<hi`.

WHEN TO USE

- Optimization problem where directly computing the optimum is hard.
- **Verification is easy**: checking whether a given answer X works is straightforward.
- Answer space is **monotonic**: feasibility flips exactly once from True to False (or vice versa).
- Phrases: "maximize the minimum", "minimize the maximum", "find the largest K such that..."
- Works on answer spaces that are real-valued (use ϵ -based termination) or integer-valued.

EXAMPLES

- Maximum minimum distance between K cows in N stalls — classic USACO problem.
- Minimize the maximum load when splitting array into K subarrays.
- Find the K -th smallest element in a sorted matrix — binary search on value.
- Allocate minimum pages such that no student reads more than X pages.
- Rope cutting — maximum rope length such that at least K pieces of that length exist.

◆ Graph Traversal (BFS / DFS)

$O(V)$

CORE PHILOSOPHY

Model the problem as a network of nodes (states) connected by edges (valid transitions). BFS explores layer-by-layer and is optimal for **unweighted shortest paths**. DFS dives deep and is ideal for **connectivity, cycle detection, topological order, and SCCs**. The art is in the modeling: recognizing that an implicit state space is actually a graph.

TIP: For grid problems, precompute the 4 (or 8) neighbor deltas — it saves repeated if-else chains.

TIP: Always mark nodes as visited before adding to queue (not when popping), to avoid re-adding.

WHEN TO USE

- Physical or implicit **grids, networks, or dependency chains**.
- BFS: shortest path in unweighted graph; multi-source BFS for distance from multiple origins.
- DFS: connected components, cycle detection, topological sort, bridges, articulation points.
- 0-1 BFS (deque): edge weights are only 0 or 1 — runs in $O(V+E)$.
- Dijkstra: non-negative weighted shortest paths — $O((V+E) \log V)$ with a min-heap.
- Bellman-Ford: negative edge weights or negative cycle detection — $O(V \cdot E)$.

EXAMPLES

- Shortest path out of a 2D maze — BFS from start cell.
- Number of connected friend groups — DFS to find connected components.
- Course prerequisites — detect cycles in a directed graph (topological sort).
- Water flow on a 3D terrain — multi-source BFS from boundary cells.
- Minimum cost network flow — augmenting paths via BFS (Edmonds-Karp).

The SMART SEARCH Family

These paradigms bridge Construct and Search. They are fundamentally search algorithms, but they exploit **problem structure and memory** to eliminate redundant work, converting exponential brute-force into polynomial solutions. Understanding *which sub-problems overlap* and *how to decompose recursively* are the core skills.

◆ Dynamic Programming (DP)

 $O(N^2)$ to $O(N)$

CORE PHILOSOPHY

Break a massive, overlapping search space into smaller sub-problems. Solve each sub-problem once, store the result (memoization / tabulation), and reuse it whenever the same sub-problem recurs. The state design is everything: a good state captures exactly the information needed to make future decisions, no more and no less.

TIP: State design checklist: (1) what varies? (2) what do you need to decide future choices? Those are your dimensions.

TIP: Space optimization: if $dp[i]$ only depends on $dp[i-1]$, reduce from $O(N^2)$ to $O(N)$ space with a rolling array.

TIP: Identify the recurrence before coding — write it mathematically first, then implement.

WHEN TO USE

- Problem asks for **maximum, minimum, or count of ways** to achieve something.
- Current decision depends on a **small number of previous states** (Markov property).
- Brute-force recursion has **exponential repeated sub-calls** — the same arguments recur.
- $N \leq 10^3$ – 10^4 ($O(N^2)$ DP) or $N \leq 10^5$ – 10^6 ($O(N \log N)$ optimized DP).
- Bitmask DP: $N \leq 20$ sets with subset intersection — models assignment/covering problems.

EXAMPLES

- 0/1 Knapsack — $dp[i][w]$ = max value using first i items with capacity w .
- Longest Common Subsequence — $dp[i][j]$ = LCS of first i chars of A and j chars of B.
- Coin Change (min coins) — $dp[amount]$ = min coins to make that amount.
- Matrix Chain Multiplication — optimal parenthesization via interval DP.
- Edit Distance (Levenshtein) — minimum insertions/deletions/substitutions.
- Bitmask DP: Hamiltonian path / Travelling Salesman at $N \leq 20$.

◆ Divide and Conquer

$O(N \log N)$ typ

CORE PHILOSOPHY

Take a problem of size N , split it into **independent, disjoint subproblems** of size $N/2$, solve each recursively, then **merge** the results. Unlike DP (overlapping subproblems + memory), D&C; subproblems do not overlap — each piece of data is processed by exactly one sub-call. The power comes from the merge step reducing an $O(N^2)$ operation to $O(N \log N)$ via the Master Theorem.

TIP: Always draw the recursion tree and verify the merge step runs in $O(N)$ — that's what gives $O(N \log N)$.

TIP: The cross-boundary case (elements from left half paired with right half) is where the interesting logic lives.

WHEN TO USE

- Problem can be split into **disjoint halves** of the same type (clean recursion).
- A merge step can combine two solved halves in $O(N)$ or $O(N \log N)$.
- The answer for the whole can be expressed in terms of answers for the two halves.
- Useful when the "cross-boundary" contribution (left meets right) can be computed efficiently.
- Often reduces $O(N^2)$ naive to $O(N \log N)$ via Master Theorem: $T(N) = 2T(N/2) + O(N)$.

EXAMPLES

- Merge Sort — split, recursively sort, merge in $O(N)$ — total $O(N \log N)$.
- Quick Sort — partition around pivot, recurse on each side.
- Count Inversions — count pairs (i, j) with $i < j$ and $A[i] > A[j]$ during merge sort.
- Closest Pair of Points — split by median x , solve each half, check strip in $O(N)$.
- Fast Fourier Transform (FFT) — polynomial multiplication in $O(N \log N)$.
- Karatsuba Multiplication — multiply large integers in $O(N^{1.585})$.

A Mental Checklist for Approaching Any Problem

Step	Action	Decision / Outcome
1	Read the Constraints First N tells you the required time complexity before you understand the problem.	$N=10^1 \blacksquare \rightarrow \text{Math}$. $N=20 \rightarrow \text{Brute/Bitmask}$. $N=10^3 \rightarrow O(N^2)$ DP. $N=10 \blacksquare \rightarrow O(N \log N)$. $N=10 \blacksquare \rightarrow O(N)$.
2	Ask the Golden Question Search for a greedy property or direct mathematical formula.	Can I make the best local choice right now with no regret? $\rightarrow$ Greedy. Direct formula? $\rightarrow$ Math.
3	Identify the Problem Structure What is the problem actually asking for?	"# of ways" / "max/min cost" over overlapping sub-problems $\rightarrow$ DP. Nodes + transitions $\rightarrow$ Graph. "Any valid config" $\rightarrow$ Constructive.
4	Start with Brute Force Code the $O(2 \blacksquare)$ or $O(N^2)$ solution first for small inputs.	Use it as a correctness oracle. Then optimize: add memoization, binary search, or a smarter data structure.
5	Verify with Small Cases & Edge Cases Test $N=0$, $N=1$, all-equal inputs, sorted/reverse-sorted arrays.	Most WA (wrong answer) verdicts come from edge cases that break assumptions embedded in your algorithm.

COMPLEXITY QUICK REFERENCE

N	Max Complexity	Paradigms to Consider
≤ 12	$O(N!)$	Complete search, permutations
≤ 20	$O(2 \blacksquare)$	Bitmask DP, subset enumeration
≤ 500	$O(N^3)$	Floyd-Warshall, matrix chain, interval DP
$\leq 5,000$	$O(N^2)$	Classic DP (LCS, edit distance), $O(N^2)$ graph
$\leq 10 \blacksquare$	$O(N \log N)$	Greedy + sort, merge sort, binary search, segment tree
$\leq 10 \blacksquare$	$O(N)$	Linear DP, BFS/DFS, two pointers, sieve
$\leq 10^4 \blacksquare$	$O(\log N)$	Binary exponentiation, math formula, binary search